Rythm Architect · **Project Summary**

CS 2114 Project 3 · Sarah Schallon, Haniya Naseer, Darsh Sharma, Miles D'Antonio · work from Nov 5 to Nov 11, 2026 · from the repository's history, README, tests and measurements

Summary

Name any song, an AI maps it, you play it. Rythm Architect is a four-lane rhythm game (keys D F J K) in the browser. A player types a real song; if nobody has mapped it yet, the game finds its YouTube video, **listens to it once** in the browser, measures every note, and a **trained beatmap model** turns it into three difficulties (Easy, Medium, Hard). Every tile is a note in the song, at the moment you hear it. The AI maps music; it never makes it.

4	~20 s	149	$0
built-in songs, plus any song a player maps	to chart a 3-minute song (WebGPU; ~90 s without)	automated tests, all passing (39 for the Java model)	per song to map: the AI runs in the player's browser

- **Model** (Java, `src/java/main`): `Queue` / `Node` (FIFO), `Note`, enums, `Song` (a note queue per difficulty) and `SongWriter`; runs on its own with a tiny driver and 39 JUnit tests.
- **View** (`frontend/`): the full game in HTML/CSS/TypeScript: a 3D stage, a music bank, favourites, the Generate flow, results, dev tools.
- **Backend** (`backend/`, Java Spring Boot, deployed on Render): songs, search, scores, YouTube lookups and osu! fan charts.
- **The biggest decision:** mapping songs with Claude (built, measured, then removed): its charts were off the music, slow (minutes) and cost about $1-3 a song. A trained mapper in the browser replaced it.
- **Still to do:** `Score` and `Main` (the controller, Darsh), connecting the game to the Java model through it, and persistent song storage on the deployed site.

1 · The product

The player's side has three parts. On the left, **Name a Song** searches every mapped song as you type, with a **Generate beatmap** button under the results and a **Favorites** drop-down. On the right is gameplay: a 3D stage, and under it the song, its cover or video, Preview, Play and the difficulty. A separate **music bank** screen lists every song A to Z.

- **Difficulty is density, holds, chords and speed**, not different songs: Easy plays the sounds that stand out most, spaced out (about 1 tile a second); Medium plays more, with holds where a sound rings (about 2); Hard plays most of them, with chords (about 4-6).
- **No sound on key presses.** The song is the sound, so a tile that isn't in the music is felt at once.
- **Songs play from YouTube's embedded player.** The game never downloads commercial audio; it hears the song once through tab sharing and keeps only the tiles and a loudness curve.

2 · Architecture (Model, View, Controller)

MODEL **Java · `src/java/main`**

`Node<T>`, `Queue<T>` (FIFO), `Note`, `NoteType`, `MusicNoteType`, `Difficulty`, `HitResult`, `Song` (one `Queue<Note>` per difficulty), `SongWriter` (a chart → a Song), `EmptyQueueException`. Standard library only; driven by `ModelDemo` and 39 JUnit tests.

CONTROLLER **`Main` (to come)**

Darsh's part, with `Score`: `processInput(lane, time)`, `checkMissedNotes(time)`, `getUpcomingNotes(count)`. Until it exists, the browser judges hits itself (`frontend/src/game/engine.ts`).

VIEW **`frontend/`**

React + TypeScript, three.js (WebGPU or WebGL) for the stage, Tone.js for sound, Rive animations. The mapping pipeline also runs here, in a worker: the beat tracker, the onset mapper and the trained mapper.

BACKEND `backend/`: Java 25, Spring Boot 4, Maven. It serves the API in `docs/api.md` (songs, search, favourites, the music bank, scores, YouTube lookups, osu! fan charts) from a JSON song bank seeded with the built-in songs. The dev mock (`frontend/mock-server/`) answers the same routes, so the view runs without Java.

3 · Timeline

Nov 5 · the start

- Repository and the frontend added (Miles): the game's menus, stage, intro, music and dev tools.
- Sarah writes the model: the first `Queue`, then its fix and `Node`, the enums and `Note`, and starts `Song`.
- The frontend moves into `frontend/` and is reorganized by area; the music bank plays songs in place.

Nov 7 · Spring Boot

- The Spring Boot project and its first endpoint preparations (Miles).

Nov 10 · the backend, Claude, and better mapping

- The full Spring Boot backend (every route of the API) with Claude charting songs in `SongMapper.java`; the model classes made to compile in package `main`.
- Deployed on Render: the backend as a Docker web service, the game as a static site.
- Claude charting reworked three times: songs split into parallel chunks with live Easy → Medium → Hard progress; a fix for the streamed answer; then "charter v2" (a song plan first, repeated sections charted alike, playability rules), then cache and cost fixes after a run timed out at about $3.
- A bug fixed: moving the window to another monitor's graphics adapter left the stage white. The stage now rebuilds itself when the GPU is lost.
- *One Dance* added as a built-in song from a human-made osu! chart.
- Better measuring: the Beat This! beat tracker (12 of 14 test tempos right, against 8) and Demucs splitting songs into drums, bass, vocals and other.
- The trained mapper, Ensomi R2, replaces Claude for placing tiles; the charting screen shows its real progress.

Nov 11 · cleanup and the deliverable

- Claude removed entirely (backend, frontend, docs). No AI runs on the server.
- Performance: gameplay re-measured and four slow spots fixed (section 7).
- Deliverable 3: the model brought to the spec's validation, `SongWriter` and a driver added, 39 JUnit tests, browser tests for the scope's bad input, and the README's Reflection, Next Steps and Attribution.

4 · How a song is mapped

1. Find	2. Listen	3. Beat	4. Sounds	5. Tiles	6. Save
The song's YouTube video (a pasted link, or a search by name).	The video plays once; the browser records the tab's sound, filed by video position. A tick measures the capture delay.	Beat This! finds the beats and downbeats: tempo, beat 0, meter.	The onset mapper times every sound to its attack (to a few ms), with its band, pitch and strength.	Ensomi R2 places the tiles; each difficulty keeps a subset; tiles snap onto the measured sounds.	Tiles and a loudness curve are saved; the audio is dropped.

How the mapping changed, and why

Version	What it did	Result
Onset mapper (start)	Rules over the measured sounds: Easy, Medium, Hard picked by how much each sound stands out.	Every tile on a sound, but the patterns felt mechanical.
Claude charter v1 (Nov 10)	The browser sent the measurements (no audio) to the backend; Claude (Opus) chose which sounds become tiles, in chunks, with live progress.	Worked, but often ran past the app's wait (minutes per song).
Claude charter v2	A song plan first (sections, repeats, lanes), matching repeats, server rules for playability, prompt caching.	Still judged worse than the rules ("off the music, awkward"); about $1-3 a song.
+ Beat This!, Demucs	A trained beat tracker for the grid; song splitting so each part's sounds are found on their own.	Tempo right on 12/14 test songs (was 8/14). Splitting took ~3 minutes and lagged on Miles's PC.
Ensomi R2 (now)	A model trained on human osu!mania charts, run in the browser (WebGPU, ONNX Runtime). It charts once; Easy and Medium keep the strongest of its tiles, spaced out, each stretch keeping its share.	"Works really well." ~20 s per song, free, deterministic. Splitting kept only as a fallback.

> **A measured fix.** On a synthetic drum groove with every hit at a known time, the onset mapper's tiles landed exactly on the hits (0 ms off) while Ensomi's came 29 ms early, likely from how its training charts were timed. Its tiles are now moved onto the measured sounds: afterwards 470 of 472 Hard tiles sat on a true hit, and 95% of Easy tiles on a beat.

If something fails, the next step carries on

- No trained model (download blocked, an old browser): the onset mapper's own charts; the dev tools say why.

- The browser can't share a tab's sound (Safari, Firefox, phones): an osu! fan chart when one exists, lined up with the video.
- The beat tracker unsure (under 50% of beats on its grid): the onset mapper's own tempo estimate.

5 · The Java model (Deliverable 3)

Class	What it does	Spec rules it follows
`Node<T>`	One link: data and the next node.	A null next ends the chain; data may be null.
`Queue<T>`	`enqueue` (a node or data), `dequeue`, `peek`, `isEmpty`, `getSize`, `clear`: first in, first out.	A null node adds nothing; empty `dequeue` / `peek` throw `EmptyQueueException` (the spec contradicted itself here).
`Note`	Letter, role in the music, length and time (seconds), lane 0-3.	No type → A; length ≤ 0 → 1; `toString` is "Type, length".
`Song`	Title, artist, duration, BPM, and a `Queue<Note>` per difficulty.	Rejects blank names and non-positive numbers; `getChart` is null when unavailable; `setChart` rejects nulls.
`SongWriter`	Builds a Song from a chart file (the built-in songs' charts are exported to `src/java/charts/`); `chartFor` picks one.	Blank title or artist, bad numbers, bad notes and unknown lines are rejected with a message; no difficulty chosen → Easy.
`ModelDemo`	The tiny driver: plays a real chart's queue front to back and shows bad input turned down.	`javac -d out src/java/main/*.java`, then `java -cp out main.ModelDemo`.

Driver output on *One Dance*, Easy: 196 of 196 notes played, in time order, queue empty at the end; four bad charts turned down with their reasons.

6 · Backend and deployment

Routes	What for
`GET /api/search`, `/favorites`, `/library`	Search as you type, the favourites drop-down, the music bank (A to Z).
`GET/PATCH/DELETE /api/songs/:id`, `POST /api/songs`	A song; renaming, favouriting, re-syncing; deleting (dev tools); saving a newly mapped song.
`POST /api/songs/:id/scores`	Saving a run's score and best.
`GET /api/youtube`, `POST /api/fan-chart`	Finding a song's video; an osu! fan chart when the browser can't listen.

- **Render:** the backend as a Docker web service (built from the repo root, so it compiles the model in `src/java/main`), the game as a static site with `VITE_API_BASE_URL` pointing at it, `CORS_ORIGINS` allowing it.

- **Settings:** `YOUTUBE_API_KEY` (searching by name), `SONGS_FILE`, `PORT`. `ANTHROPIC_API_KEY` is no longer used and can be deleted.

7 · Quality: tests, bugs, performance

Tests	Count	What they cover
Model (JUnit, `src/java/test/main`)	39	Queue 13, Node 5, Note 7, Song 7, SongWriter 7: the spec's test plan, good and bad cases, plus the real built-in charts.
Backend (JUnit, Spring)	10	Every route against a fresh song bank; name matching; the osu! port.
Frontend unit (Vitest)	78	Game rules, the onset mapper (with a note-finding benchmark), the beat grid, the trained mapper's thinning and timing, sync, YouTube, the bank, performance helpers.
Browser end to end (Playwright)	22	The whole game: intro, search, Generate (with a stand-in YouTube player), listening and syncing, fan charts, the bank, favourites, playing, failing, dev tools, phones, the scope's bad input, and the trained mapper failing.

Bugs found and fixed (selected)

- **White stage after moving monitors** (Vivaldi): the GPU was taken away; the stage now rebuilds itself and replays its state.
- **Claude's streamed answer failed in Spring** ("No converter … ndjson"), and a wrapped error hid the real reason; both fixed before Claude was removed.
- **Beat grid unused** on some songs (24% confidence): the dev tools now show what the beat tracker found and whether it was used.
- **Ensomi's tiles 29 ms early**: measured against known hits and corrected (section 4).
- **The charting screen was unclear** (meters empty for a minute, then all at once): now real progress, then each difficulty in turn.

Performance (Nov 11)

Gameplay was measured with the autoplay bot. The game code itself hadn't slowed (the version before the GPU fix ran the same), and denser Ensomi charts weren't heavier. Four slow spots were found and fixed anyway:

Fix	Effect
The YouTube video's dock measured the page layout every frame.	Generate, then play, same tab: page 65-76 → 88-91 fps; long frames 30-72 → 12-13 per 15 s; stage 90-107 → 107-116 fps.
Judgement words asked the browser for their animations on every hit (a style recalculation).	
Every frame, the state of every note in the song was packed and sent to the stage, which walked all of them.	Now only the notes near the hit line; work no longer grows with song length.
Each Generate read the whole cached 120 MB model into memory.	It only checks the cache; new files stream straight into it.

8 · Changes from the spec (short form)

The README's Reflection has the full table: what the spec said, what we built, why, and what prompted it.

- **SongWriter:** Claude-generated songs → building a Song from a chart the trained mapper made (Claude: worse charts, slow, costly).
- **Queue:** empty `dequeue` / `peek` throw `EmptyQueueException`; `enqueue(T)` added beside `enqueue(Node)`.
- **NoteType:** note letters A-G, not NORMAL/HOLD/SLIDE (holds are long notes; lanes say how a note is played).
- **Model/View boundary:** the view loads a song's chart once (to draw tiles down the whole track) but works only with the notes near the line each frame.
- **Wrong-time presses:** ignored rather than penalized (missing the tile already costs); undefined keys ignored, as specified.
- **Controller:** `Main` not yet written; the browser judges hits until it is.

9 · What's next

#	Item	Owner
1	`Score`: `addHit(HitResult)`, `addMiss()`, `getAccuracy()`, … with `ScoreTest`.	Darsh
2	`Main`: `processInput`, `checkMissedNotes`, `getUpcomingNotes` over `SongWriter.chartFor(...)`'s queue, with `MainTest`.	Darsh
3	Spring Boot routes for `Main` and a client for them in the game.	Darsh, Miles
4	The game plays through the controller (the browser's own rules kept as the offline fallback).	Miles
5	Tests for `Score` and `Main`, an end-to-end test through the controller, beta testing.	Haniya
6	Persistent song storage and a YouTube API key on Render.	Miles
7	Review the model changes (`Queue`, `Note`, `Song`, `SongWriter`).	Sarah

Change feature beyond the MVP: Practice mode (loop a section, optionally slower). Predicted to touch `Song` (a new `getChartBetween(Difficulty, double, double)`), `Main` (scaled time, `startPractice(...)`), `Score` (practice runs not saved) and the view's playback; `Queue`, `Node`, `Note`, the enums and `SongWriter` should not change.

10 · Credits

- **Team:** Sarah Schallon (model: Queue, Node, Note, enums, Song), Haniya Naseer (test plan, testing), Darsh Sharma (Score, Main), Miles D'Antonio (view, backend, deployment, SongWriter; with Claude Code). The README's Attribution table has every class and test file.
- **GenAI:** the spec started from a GenAI template; most code outside Sarah's classes was written with Claude Code at Miles's direction, play-tested and checked by tests. Claude also mapped songs at run time until Nov 11.
- **Models:** Ensomi R2 (MIT) with @ensomi/runtime (Apache-2.0); Beat This! (CPJKU, MIT); Demucs v4 (Meta, MIT); ONNX Runtime Web (MIT).
- **Songs:** *Ink and Fury* and *The Restless Stacks* were composed for SchmollCrossword (MIT); *One Dance*'s tiles are the osu! chart "Normal" by Alkin098 (the song plays from YouTube). Full credits in `NOTICE.md`.